

— AN END-TO-END BIG DATA PIPELINE —

Detecting IoT Malware at Network Scale.

DATA ENGINEER

IVAN ILYICHEV

PIPELINES · AUTOMATION

ML SPECIALIST

KIRILL SHUMSKY

DISTRIBUTED ML · SPARK

DATA SCIENTIST

NIKITA ZAGAINOV

EDA · DASHBOARDS

TESTER / TECH WRITER

DMITRY TETKIN

DOCUMENTATION · QA

BRIEF

Can we classify malicious IoT network traffic from raw Zeek connection logs?

25.0M

NETWORK CONNECTIONS

12

IOT CAPTURE SOURCES

7

MALWARE BEHAVIOUR CLASSES

0.997

RANDOM FOREST · F1

A Hadoop / Spark pipeline that ingests 25M Zeek records from PostgreSQL, partitions them in Hive, and trains distributed multiclass classifiers for seven malware behaviours.

STAGES OF WORK

From CSV to dashboard, in four reproducible stages.

STAGE 01 Ingestion PostgreSQL + Sqoop import to HDFS as Parquet/Snappy.	STAGE 02 Storage & EDA Hive partitioned + bucketed tables. Seven Tez/HiveQL insights.	STAGE 03 Predictive Analytics Spark ML pipeline — RF and Multinomial LR with grid-search CV.	STAGE 04 Dashboard Apache Superset — data characteristics, EDA insights, model results.
--	--	---	--

CLUSTER

Innopolis Hadoop · 3 nodes · YARN · Hive + Tez · Spark on YARN.

STACK

Python 3 · PySpark · Sqoop · Beeline · psycopg2 · Apache Superset.

REPRODUCIBILITY

Single entry point `main.sh` — every stage is idempotent and re-runnable.

01

Data Collection & Ingestion.

12 CAPTURE FILES. TWO RELATIONAL TABLES. ONE SQOOP IMPORT.
PARQUET ON HDFS, SNAPPY-COMPRESSED, READY FOR HIVE.

SOURCE

Real IoT-device captures from the Aposemat IoT-23 corpus.

12 selected captures of *Mirai*, *Hide-and-Seek*, *Hakai*, *Torii*, *Trojan*, *Okiru*, *Kenjiro* and benign baselines — converted to Zeek conn.log format with binary & multi-class malware labels.

VOLUME & SHAPE

ROWS 25,011,247 network connections

CAPTURES 12 distinct IoT-device traces

FEATURES 23 per-connection Zeek fields

TARGET 7-way label (Benign + 6 attack types)

1.1 RELATIONAL SCHEMA

Two tables, one foreign key.

CAPTURES

capture_id int (PK)
capture_name varchar(128)
source_file varchar(255)
created_at timestamptz

NETWORK_CONNECTIONS

connection_id bigint
capture_id int (FK)
ts timestamptz
id_orig_h / id_resp_h inet
proto · service · conn_state varchar
orig_bytes / resp_bytes bigint
orig_pkts / resp_pkts bigint
history · duration · ports ...
label · detailed_label varchar

Stage table stg_network_connections ingests raw text via COPY; type casts (inet, to_timestamp, regex-cleaned ints) happen in the SQL transform.

1.2 INGEST PIPELINE

Four steps. One reproducible load.

STEP 01 Collect gdown pulls 12 capture CSVs from Google Drive into data/.	STEP 02 Load COPY into staging, then transformed insert into typed PostgreSQL tables.	STEP 03 Validate 7 SQL assertions on row counts, label coverage and FK integrity.	STEP 04 Sqoop Both tables exported to HDFS as Parquet + Snappy via single-mapper queries.
--	--	--	--

1.3 STORAGE BENCHMARK

We compared four format / codec pairs.

FORMAT	CODEC	WRITE (S)	SCAN (S)
parquet	snappy	17.7	6.76
parquet	gzip	18.0	6.64
avro	snappy	17.0	9.32
avro	bzip2	19.0	8.36

STAGE 01 · DATA COLLECTION & INGESTION

1.4 SQOOP COMMAND (EXCERPT)

```
# network_connections → HDFS
sqoop import \
  --connect jdbc:postgresql://hadoop-04/team30_projectdb \
  --compression-codec snappy --compress \
  --as-parquetfile \
  --target-dir /user/team30/project/
  warehouse/network_connections \
  --m 1 \
  --query "SELECT ... FROM
  network_connections WHERE \${CONDITIONS}"
```

Single mapper avoids skew on the auto-incremented PK; UTC-formatted timestamps survive the round-trip.

02

Storage & EDA.

EXTERNAL PARQUET → BUCKETED CAPTURES + LABEL-PARTITIONED CONNECTIONS. SEVEN HIVEQL QUERIES RUN ON TEZ; RESULTS MATERIALISED AS PARQUET RESULT TABLES.

2.1 OPTIMISED TABLES

Two physical layouts — one for joins, one for label scans.

EXT	<code>captures_ext</code> & <code>network_connections_ext</code> over Sqoop output.
BUC	<code>captures_bucketed</code> — CLUSTERED BY (<code>capture_id</code>) INTO 4 BUCKETS.
PART	<code>network_connections_part</code> — PARTITIONED BY (<code>label</code>).
FMT	All managed tables stored as PARQUET on HDFS.

WHY

Label is the dominant filter in EDA & ML — partitioning by it lets every analytic query prune to a fraction of the data.

2.2 ENGINE & SETTINGS

HiveQL on Tez with dynamic partitioning.

```
SET hive.execution.engine=tez;
SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;
SET hive.enforce.bucketing=true;

-- bucketed insert
INSERT OVERWRITE TABLE captures_bucketed
SELECT capture_id, capture_name, source_file,
       CAST(created_at AS TIMESTAMP)
FROM captures_ext;

-- partitioned insert (label dynamic)
INSERT OVERWRITE TABLE network_connections_part
PARTITION (label) SELECT ... , label
FROM network_connections_ext;
```

7 EDA queries materialise `qN_results` Parquet tables and stream CSV exports through Beeline for the dashboard.

01

A SKEWED BUT RICH TARGET

Benign and Malicious dominate — six rare attack types remain useful.

The corpus is neither balanced nor degenerate: 35% benign, 28% generic Malicious, 23% DDoS, 14% port-scan, plus three rare classes that the multiclass models still need to handle.

Benign label = "Benign"		35.11%
Malicious label = "Malicious"		28.21%
DDoS label = "Malicious DDoS"		23.10%
Port scan PartOfAHorizontalPortScan		13.54%
C&C / Attack / FileDownload 3 rare classes combined		0.05%

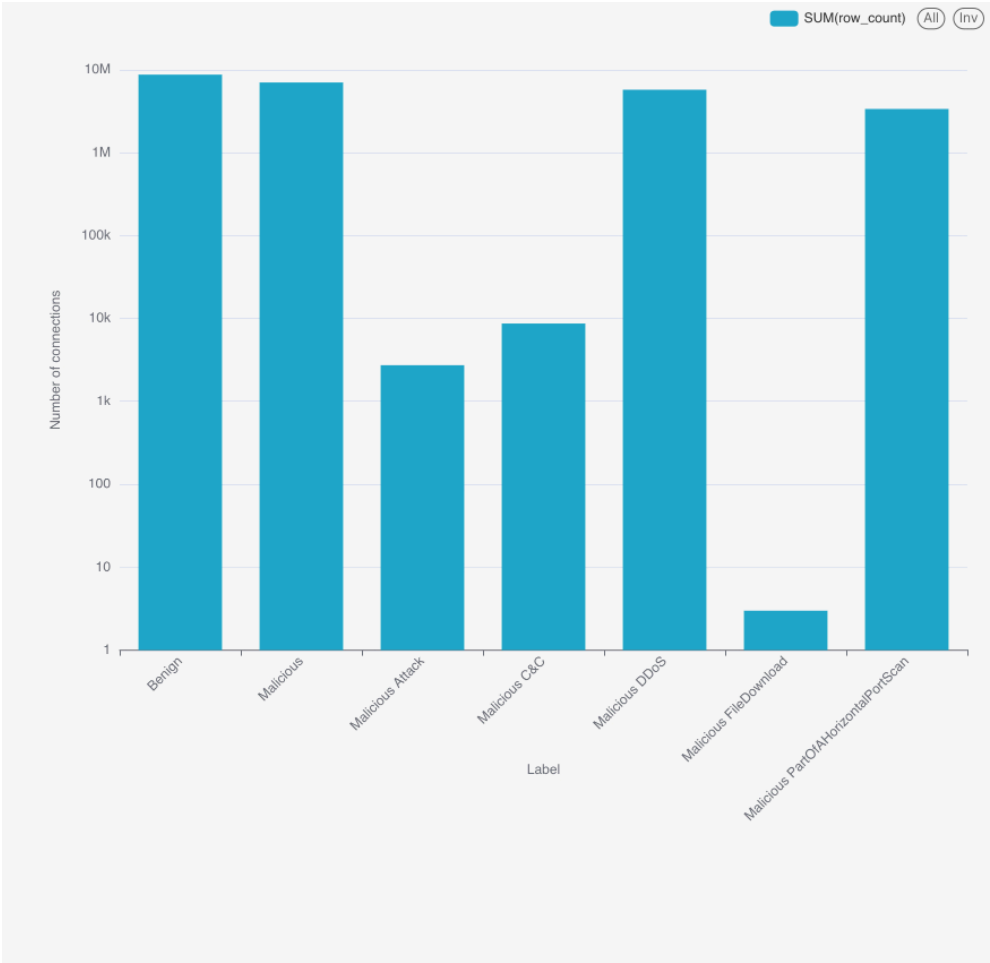


FIG. 01 · CLASS SHARE OF NETWORK_CONNECTIONS_PART. THE LONG TAIL OF RARE ATTACK TYPES IS THE KEY REASON THE PROJECT IS TREATED AS MULTICLASS, NOT BINARY.

02

ALMOST EVERYTHING IS TCP

Protocol mix per label.

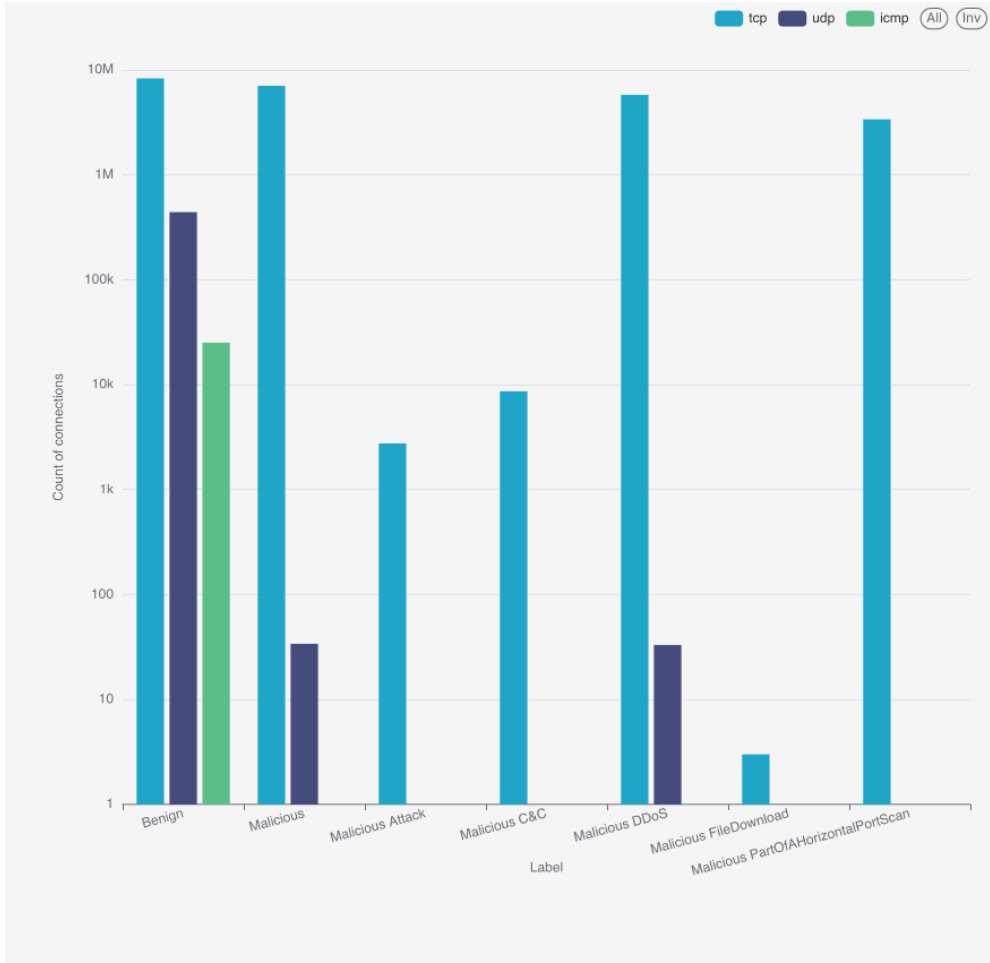


FIG. 02 · UDP APPEARS ALMOST ONLY FOR BENIGN DNS AND ICMP-ONLY FLOWS; NEARLY EVERY MALICIOUS CONNECTION IS TCP.

05

FAILED HANDSHAKES EVERYWHERE

Connection state per label.

Label	Dominant Conn_State	Share
Benign	S0 (no reply)	99.3%
Malicious	S0	99.7%
Malicious DDoS	OTH / RSTOS0	99.99%
Malicious C&C	S0 + S3	88%
Port scan	S0	100%

A SYN-only flood (S0) is the universal signature — benign traffic on this network is dominated by IoT devices repeatedly failing to reach a server.

03

VOLUMETRIC ATTACKS VS. SILENT SCANS

Average bytes per connection by label.

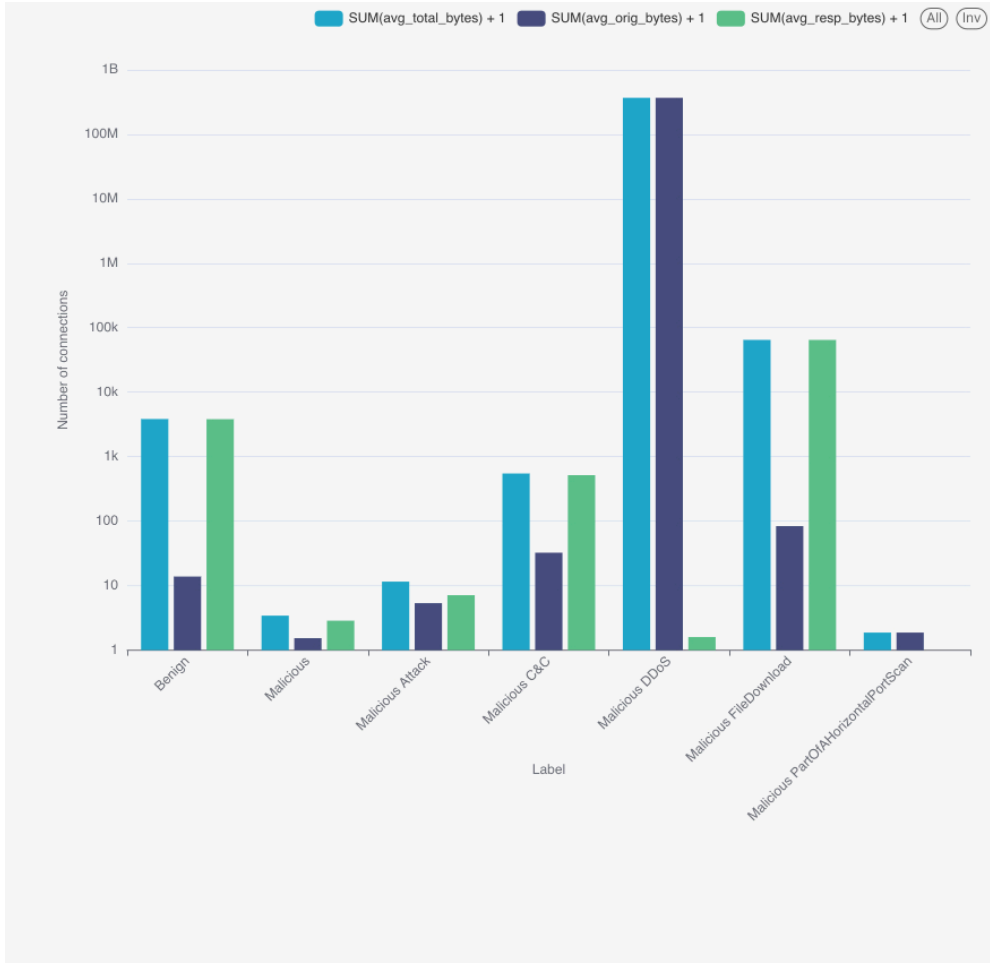


FIG. 03 · DDoS AVERAGES $\approx 3.7 \times 10^8$ ORIG BYTES / FLOW – SIX ORDERS OF MAGNITUDE OVER BENIGN. PORT-SCAN AND GENERIC MALICIOUS CARRY < 1B PER CONNECTION ON AVERAGE.

07

PACKET-RATE SIGNATURES

Average packets per connection.

LABEL	AVG ORIG PKTS	AVG RESP PKTS
Benign	3.90	0.006
Malicious	1.10	0.033
Malicious DDoS	48.10	0.0004
C&C	495.4	8.15
Attack	5.34	3.40
Port scan	4.00	0.00

Long-lived C&C sessions stand out: ~500 pkts out, with a real (8 pkt) reply — very different from one-shot scan or flood traffic.

04

CAPTURES CLUSTER BIMODALLY

Almost all malicious or almost all benign — nothing in between.

8 of the 12 captures are > 79% malicious; 4 captures are < 11% malicious. There is no middle ground — each PCAP was deliberately recorded around a single behaviour.

CAPTURE	TOTAL FLOWS	MALICIOUS RATIO
Capture-60	3,581,028	99.9%
Capture-9	6,378,293	99.6%
Capture-48	3,394,338	99.9%
Capture-3	156,103	97.1%
Capture-1	1,008,748	53.5%
Capture-44	237	11.0%
Capture-20 / 22 / 42	~3,500 each	<1%

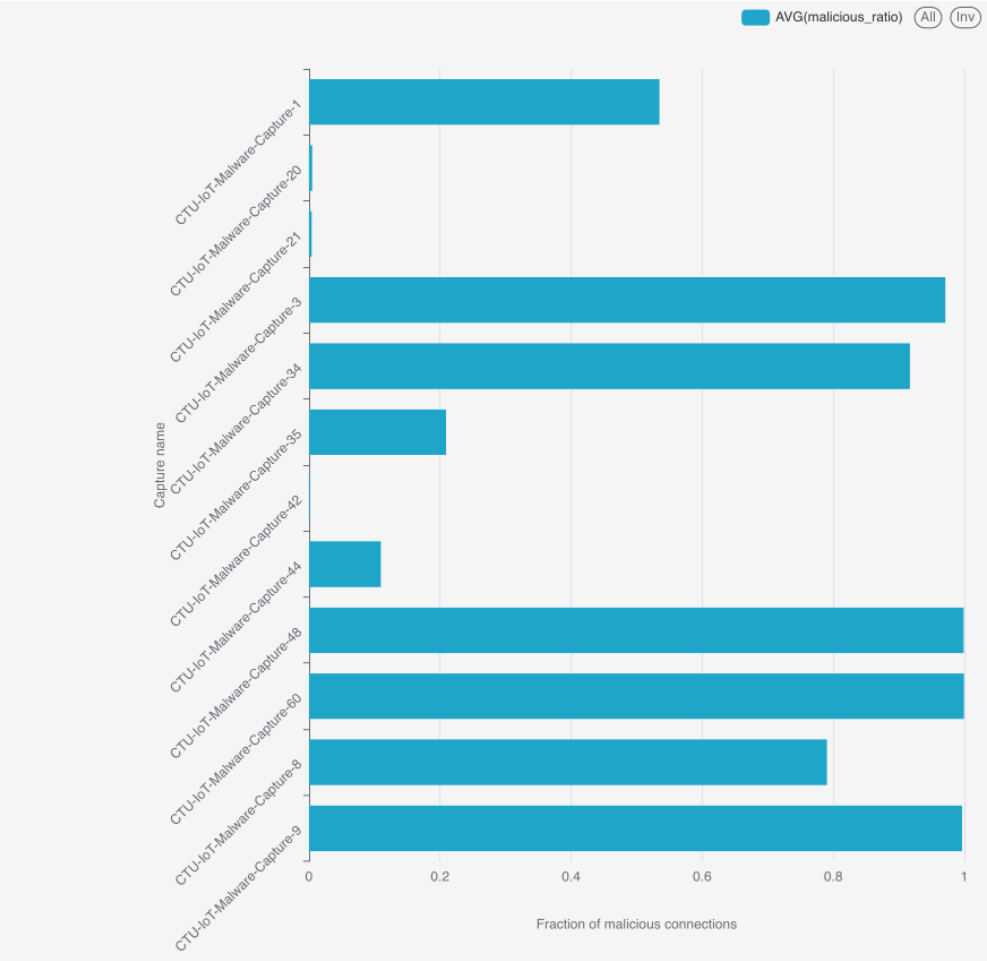


FIG. 04 · TREAT CAPTURES AS STRATA, NOT INTERCHANGEABLE ROWS – RANDOMLY SPLITTING AT THE ROW LEVEL CAN LEAK WHOLE DEVICES INTO BOTH TRAIN AND TEST.

06

TELNET IS STILL THE WOUND

Port 23 is the single most attacked destination.

Generic Malicious traffic (4M flows) and the entire port-scan class (3.4M flows) target Telnet (tcp/23) — the classic Mirai-family entry point. DDoS is bimodal between 62336 (Mirai-style C&C echo) and HTTP 80.

LABEL	PORT	FLOWS
Malicious	23	4,055,327
DDoS	62336	3,578,391
Port scan	23	3,386,119
Malicious	81	2,571,979
DDoS	80	1,125,806
DDoS	992	1,008,351
C&C	6667	6,706

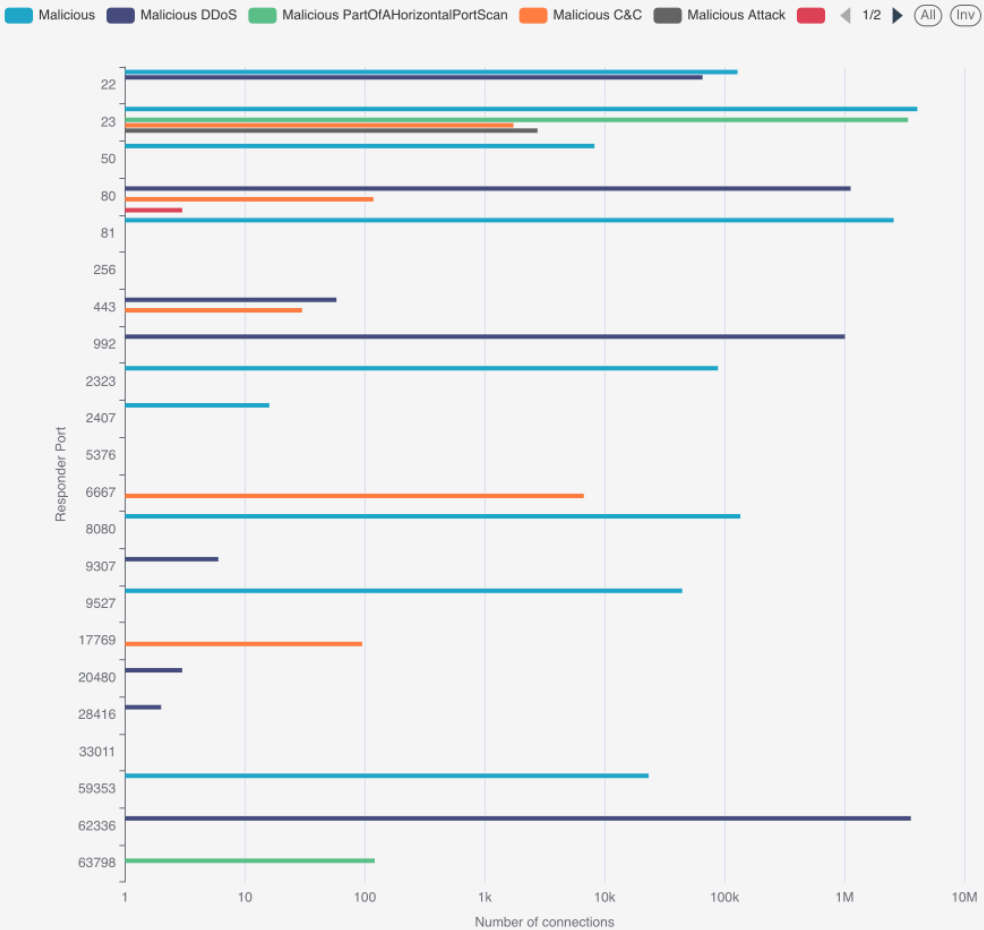


FIG. 06 · TARGET-PORT HISTOGRAMS PER NON-BENIGN LABEL. PORT 23 DOMINANCE IS A STRONG, LEARNABLE SIGNAL – IT IS ONE OF THE HIGHEST-IMPORTANCE FEATURES IN THE TRAINED RANDOM FOREST.

03

Predictive Analytics.

A 7-CLASS SUPERVISED PROBLEM ON 25M ROWS. CYCLICAL TIME
ENCODING, CHISQ FEATURE SELECTION, TWO CROSS-VALIDATED
MODELS – ALL DISTRIBUTED ON YARN.

3.1 PIPELINE

From raw Hive rows to a single scaled feature vector.

STEP 01 Time encode Custom CyclicalTimeTransformer — sin/cos for month, day, hour, minute, second.	STEP 02 Impute Fill nulls per dtype: "unknown" for strings, 0/false for numerics, drop tunnel_parents.	STEP 03 One-hot StringIndexer + OneHotEncoder for proto, service, conn_state, history.	STEP 04 Select ChiSqSelector(fpr=0.1) over categorical block — keeps only label-correlated dummies.	STEP 05 Assemble + scale VectorAssembler + StandardScaler → final features column.
--	--	--	---	--

NUMERIC FEATURES

duration · orig_bytes · resp_bytes · missed_bytes · pkts (orig/resp) · ip_bytes · ports · year · local_orig · local_resp.

CATEGORICAL (AFTER CHISQ)

proto · service · conn_state · history — selected on training fold only to avoid leakage.

SPLIT

60% / 40% random split, seed = 42. Final train/test persisted as Parquet on HDFS for both training and evaluation.

3.2 TWO MODEL FAMILIES

A non-linear baseline against a linear baseline.

M1 **Random Forest** — numTrees $\in \{30, 60\} \times$ maxDepth $\in \{4, 6\}$. Bagging handles the long tail of rare classes naturally.

M2 **Multinomial Logistic Regression** — regParam $\in \{0.01, 0.1\} \times$ elasticNet $\in \{0.0, 0.5\}$, maxIter = 50.

CV 2-fold cross-validation, parallelism = 3, optimised on multiclass f1.

CLUSTER

Each spark-submit: 6 executors $\times$ 3 cores $\times$ 6 GB on YARN.
Train data .persist(MEMORY_AND_DISK) to amortise CV folds.

3.3 BEST RANDOM FOREST

Hyperparameters chosen by CV.

HYPERPARAMETER	BEST VALUE
numTrees	60
maxDepth	6
impurity	gini
featureSubsetStrategy	auto

3.4 BEST LOGISTIC REGRESSION

HYPERPARAMETER	BEST VALUE
regParam	0.01
elasticNetParam	0.0
family	multinomial
maxIter	50

3.5 COMPARISON

Both models clear **F1 = 0.99** — Random Forest slightly ahead.

After `ChiSqSelector` trims uninformative dummies and `StandardScaler` harmonises feature scales, the multinomial linear baseline is genuinely competitive on this problem.

MODEL	ACCURACY	PRECISION	RECALL	F1
Random Forest	0.9974	0.9974	0.9974	0.9973
Logistic Regression	0.9943	0.9939	0.9943	0.9941

Metrics computed via `PySpark MulticlassClassificationEvaluator`; F1 weighted by class support.

3.5 VISUAL RANKING

Bars scaled by F1 score.



RF reaches **F1 = 0.997** on a 7-class, 25M-row problem; LR sits 0.003 behind at **0.994** — a strong baseline rather than a straw man.

Predictions persisted to HDFS as CSV (`project/output/modelN_predictions/`) and replayed by `ml_evaluation.py` — evaluator is decoupled from training.

04

Dashboard.

A SINGLE SUPERSET DASHBOARD BACKED BY THE SEVEN Q*_RESULTS
HIVE TABLES AND THE MODEL EVALUATION CSV. PUBLIC,
REPRODUCIBLE, REFRESH-ON-DEMAND.

4.1 DASHBOARD SECTIONS

From raw scale to model verdict, in five panels.

A	Data characteristics — 25M rows, 23 features, 7 classes, 12 captures.
B	Class balance (q1) — donut and percentage list.
C	Behaviour signatures (q2, q3, q5, q7) — protocol, bytes, conn_state, packets.
D	Top targets (q6) — port histograms per attack type.
E	Capture strata (q4) — bimodal maliciousness ratios.
F	Model performance — RF vs. LR comparison from <code>evaluation.csv</code> .

4.2 WIRING

Backed entirely by managed Hive tables.

```
-- Superset → Hive 2 (HiveServer2)
jdbc:hive2://hadoop-03.uni.innopolis.ru:10001
  USE team30_projectdb;

-- One dataset per chart
SELECT label, row_count, share_fraction
  FROM q1_results;

SELECT model, accuracy, precision, recall, f1_score
  FROM evaluation_csv;
```

REPRODUCIBILITY CONTRACT

Re-run `main.sh` on the cluster — every Hive result table and the evaluation CSV are rebuilt; the dashboard refreshes against them with no manual edits.

Heavy work stays manual; the per-commit loop stays fast.

The full pipeline downloads ~3.39 GB and competes for YARN — never on a push. CI runs only what's safe and quick.

LAYER	TRIGGER	WHAT
1 · Lint / unit	push / PR	ShellCheck, ruff + pylint, pdf ¹ atex, bats unit
2 · Cluster smoke	manual	Postgres, HDFS, Hive, Spark eval re-run
3 · Full pipeline	manual	bash main.sh end-to-end

WORKFLOW FILE

.github/workflows/lint.yml

Four jobs in parallel, ~1m wall-time.

All checks have passed

4 successful checks

✓

lint / Build report.pdf (push)

Successful in 1m

Details

✓

lint / ShellCheck (*.sh) (push)

Successful in 17s

Details

✓

lint / bats unit tests (no cluster) (push)

Successful in 6s

Details

✓

lint / ruff + pylint (*.py) (push)

Successful in 33s

Details

Green: ShellCheck · ruff + pylint · bats unit · Build report.pdf.

Some checks were not successful

2 successful, 1 failing, and 1 in progress checks

✗

lint / ruff + pylint (*.py) (push)

Failing after 30s

Details

🔄

lint / Build report.pdf (push)

In progress - This check has started...

Details

✓

lint / ShellCheck (*.sh) (push)

Successful in 17s

Details

✓

lint / bats unit tests (no cluster) (push)

Successful in 5s

Details

Earlier run on the same PR — ruff + pylint caught a regression in seconds; other three jobs delivered partial feedback in parallel.

WHY A SECOND LAYER

Postgres, HDFS, Hive and Spark live behind the Innopolis network.

Cloud runners can't see them, so the cluster smoke suite is a hand-fired follow-up that reuses what's already loaded — no gdown, no retraining.

PG	7 SQL assertions: 12 captures, ≥24M rows, all 7 labels, FK integrity.
HDFS	project/warehouse sizes match Stage 1; project/data/{train,test} present.
HIVE	Full table inventory + q1.hql rerun and row-count diff against output/q1.csv.
SPARK	ml_evaluation.py rerun over persisted predictions — no retraining.

CLUSTER • RUN OUTPUT

19 / 19 unit tests green on hadoop-01.

```
team30@hadoop-01:~/project30$ cd ~/project30 && bash scripts/run_tests.sh unit 2>&1

=== Running unit suite (2 file(s)) ===
1..19
ok 1 main.sh exists at repo root
ok 2 all four stage scripts exist and start with bash shebang
ok 3 every stage script uses set -euo pipefail
ok 4 all 7 HQL EDA queries are present
ok 5 Hive warehouse DDL exists
ok 6 PostgreSQL DDL and test queries exist
ok 7 ML pipeline scripts are all present
ok 8 .gitignore excludes data/, secrets/, .venv/
ok 9 secrets/ contents are not tracked by git
ok 10 evaluation.csv has both models and four metrics
ok 11 Random Forest F1 in evaluation.csv is above 0.99 (regression guard)
ok 12 all 7 EDA CSVs exist and are non-empty
ok 13 all 7 EDA charts exist and are non-empty
ok 14 storage-benchmark CSV is committed
ok 15 model directories are committed for both models
ok 16 final report PDF is committed and non-trivial
ok 17 no leftover Citus references in report.tex
ok 18 no '737 MB' regression in report.tex (must be 3.39 GB)
ok 19 report.tex points at the new repo URL

All selected tests passed.
team30@hadoop-01:~/project30$
```

Direct run from team30@hadoop-01: file-layout invariants, every stage uses set -euo pipefail, evaluation.csv schema, RF F1 regression guard, and the three report.tex regression checks (no Citus, raw CSV size pinned at 3.39 GB, new repo URL).

Same runner, two destinations. Tests gated by require_cmd / require_secret — on the cluster they validate the live warehouse, on a laptop the cluster tests just skip.

CLUSTER RUNNER

```
bash scripts/run_tests.sh smoke
bash scripts/run_tests.sh smoke 02_postgres
```

WHAT WE BUILT

A reproducible Hadoop pipeline for IoT malware classification — from CSV to model to dashboard.

SCALE	25M Zeek rows ingested to Hive in < 20 s with Parquet/Snappy.
SIGNAL	7 EDA insights expose class skew, port-23 dominance, S0-flood signature.
MODEL	Both RF and LR clear F1 = 0.99 across 7 classes; RF leads by ~0.003.
SHIP	Single main .sh entry point + public Superset dashboard.

REFLECTIONS & NEXT STEPS

What we'd change in v2.

LESSONS

Capture-level bimodality (q4) is a real risk for row-level random splits — group-aware splitting per `capture_id` is the right next iteration. RF leads LR by only ~0.003 F1, so the next ensemble worth trying is Gradient-Boosted Trees rather than more tuning of either head.

NEXT STEPS

Gradient-boosted trees · per-capture cross-validation · streaming variant via Spark Structured Streaming on the same Hive partitions · per-class precision/recall in the dashboard.

FINAL POSITION

Big-data tools were the right hammer here — Hive partitioning + Spark CV made the 25M-row, 7-class job tractable on a 3-node cluster.

—— THANK YOU

Questions welcome.

A Hadoop / Spark pipeline that turns 25 million raw IoT connections into a single, reproducible verdict — and a dashboard maintainers can actually read.